מבחן בית במערכות הפעלה- ציון 91



שאלה 1
הושלם
14.00 נקודות
מתוך 20.00
סימון שאלה

הוצע שינוי מדיניות תזמון:

1. כל תהליך מקבל 100 נקודות בזמן יצירתו.

2. לכל תהליך שיצא ממצב רץ (running state) מורידים עד 50% מנקודות שיש ברשותו בהתאם לאחוזי ניצול הקוונטום שקיבל, שהוא אלפית השנייה. זמן ריצה בפועל נמדד עם דיוק של מיליונית השנייה.
לדוגמא, תהליך שהיו ברשותו 80 נקודות ורץ קוונטום שלם ישלם 40 נקודות, אם הוא רץ חצי קוונטום, ישלם 20 נקודות. אם הוא רץ פחות ממיליונית השנייה אז זמן ריצה שלו שווה לאפס.

3. הנקודות שהורדו מהתהליך מתחלקות בין כל התהליכים (כולל גם אותו) ביחס למספר נקודות שברשותם. לדוגמה, אם הורדו 20 נקודות ויש שני תהליכים שברשות האחד 60 והשני 120 אז האחד יקבל ...6.66 והשני יקבל ...13.33.

4. המתזמן בוחר את התהליך עם הכי הרבה נקודות.

5. אחרי סיום של התהליך לוקחים ממנו 100 נקודות בחזרה, אם יש לו יותר, היתרה מתחלקת בין כל תהליכי המערכת יחסית לפי נקודות שברשותם. אם יש לו פחות מ 100 נלקח מהם נקודות, בהתאם למס' נקודות שברשותם.

א. (4 נק')
האם אפשר לקבוע כמה נקודות יש בסה"כ אצל כל התהליכים שקיימים במערכת? אם כן, ציונו כמה.

ב. (4 נק')
המדיניות הנ"ל יכולה להביא להרעבה. תביאו דוגמא לכך

ג. (4 נק')
תציעו שינוי קל ששומר על המדיניות בכללותה, אבל לא גורם להרעבה.

ד. (4 נק')
המדיניות הנ"ל אינה הוגנת ביחס להרצת אפליקציות שונות. תסבירו מדוע.

ה. (4 נק')
האם יכול להיות מצב שתהליך מסתיים כשברשותו פחות מ 100 נקודות? הביאו דוגמא לכך או הביאו הוכחה נגדית.

<u>סעיף א:</u>
רשום שכל תהליך מקבל בשעת יצירתו 100 נקודות , אז סה"כ אצל כל התהליכים במערכת יהיו:
100 נקודות * מספר התהליכים = מספר כולל של נקודות במערכת עבור כל התהליכים

<u>סעיף ב:</u>
תתכן הרעבה במדיניות הנ"ל כיוון שתהליך שרץ וניצל קוואנטום מלא ישאר עם מעט נקודות לאחר סיום הריצה ולכן יקבל פחות נקודות מהאחרים מה שיגרום למצב שיקח הרבה זמן עד שהוא יהיה התהליך עם הכי הרבה נקודות ולכן הוא יורעב.
וכם שכאשר נוצרים תהליכים חדשים הם יהיו בעדיפות גבוהה יותר כי יש להם 100 נקודות וכך יהיה מצב של הרעבה של תהליכים שכבר רצו ויש להם פחות מ 100 נקודות.

<u>סעיף ג:</u>
בסוף ריצה של תהליך הנקודות שנשארו לו לאחר לקיחת 50% יחולקו באופן שווה בין כולם.

<u>סעיף ד:</u>
המדינויות אכן אינה הוגנת ביחס להרצת אפליקציות שונות כיוון שישנם אפליקציות עם תהליכים כבדים שמנצלים את רוב זמן המעבד מול אפליקציות עם תהליכים קלים של פעולות IO וכך תמיד תהיה עדיפות לתהליכים עם הכי הרבה נקודות - רצים הכי קצת זמן ..

<u>סעיף ה:</u>
לפי המדינויות כל תהליך מקבל 100 נקודות בהתחלה ותמיד ילקח התהליך שיש לו הכי הרבה נקודות.
ייתכן מצב שבו יש לתהליך פחות מ 100 נקודות כי מספיק שיש עוד תהליך אחד חוץ מהתהליך נוכחי ונניח אפילו הוא רץ לכמה שניות ומתוך אורך הקוואנטו אז כבר הוא פחות מ 100 נקודות והנקודות שירדו לו מתחלקות באופן יחסי בינו ובין התהליך הנוסף ולכן הא נשאר עם פחות מ 100 נקודות.
אבל גם ייתכן מצב שבו יש לתהליך תמיד 100 נקודות - למשל אם יש רק תהליך אחד שרץ במערכת אז כשהוא מסיים לרוץ אז מופחתים לו הנקודות ביחס לזמן הריצה ואז יש לחלק את הנקודות האלו בין כל התהליכים אבל כיוון שהוא התהליך היחיד אז הוא יקבל את כל הנקודות שלו חזרה - סהכ 100 נקודות

הערה:			
סעיף א'	4	/	4
סעיף ב'	4	/	2 זאת לא הרעבה, אלא חוסר הגינות, יצבור עם הזמן
סעיף ג'	4	/	2 זאת מדיניות שונה לגמרי שלא נותנת מספיק העדפה לאינטרקטיביים
סעיף ד'	4	/	4
סעיף ה'	4	/	2 מדובר שהוא הסתיים ויש לו פחות מ 100 לפני שמשלם, אחרת זה מצב טבעי.(כתוב בשאלה: מסתיים... משלם)

שאלה 2
הושלם
20.00 נקודות
מתוך 20.00
סימון שאלה

נתון מחשב עם זיכרון פיזי 64MB עם מיני-מערכת הפעלה שתופסת 16MB זיכרון. המערכת הנ"ל מריצה תהליך אחד ויחיד בלבד. גודל דף שווה ל 1KB. כדי לחסוך בגודל טבלאות הדפים, המערכת הנ"ל מאפשרת לכל תהליך מרחב זיכרון בגודל התואם לדרישותיו. המערכת מריצה כרגע תהליך עם דרישת זיכרון של 4MB.

א. (6 נק')
איזה רכיב נדרש בתוך מעבד כדי לדעת מהו מרחב הזיכרון שהוקצה לתהליך שרץ כרגע?

ב. (4 נק')
האם ייתכן דפדוף במערכת הנ"ל במצב הנתון? נמקו את תשובתכם.

ג. (6 נק')
אם טבלת הדפים במערכת הנ"ל מכילה רק את המידע הרלוונטי לאותה מערכת, מה הוא הגודל המינימאלי של טבלת הדפים של התהליך?
(* הניחו שגודל המידע הנוסף הנלווה מעבר למספר מסגרת ברשומה הוא BYTE אחד).

ד. (4 נק')
אם יוחלט לאפשר הרצת מספר תהליכים במקביל במערכת הנ"ל ומעבד תומך בזיכרון וירטואלי, אלו מידעים צריך לשמור בטבלת הדפים?

<u>סעיף א:</u>
MMU

<u>סעיף ב:</u>
יש דפדוף רק אם השטח שהפרוסס רוצה הוא גדול מהשטח שיש בזיכרון, ולכן לא יהיה דפדוף.
אך ייתכן דפדוף אם משתמשים באלגוריתם שלא טוען את כל התהליך מראש לזיכרון.

<u>סעיף ג:</u>
יש לנו בשורה 22 ביטים שהם בקירוב 3 בתים +1 מידע נלווה סה"כ 4 בתים
ולכן יש לנו 256 שורות בדף(1024/4) ולכן 2^22/2^8 = 2^14

<u>סעיף ד:</u>
המידעים שצריך לשמור בטבלה הדפים היא:
caching disabled,referenced, modified, protection , present/absent, page frame number

שאלה 3
הושלם
15.00 נקודות
מתוך 15.00
סימון שאלה

במערכת קיימים 4 תהליכים: P_1, P_2, P_3, P_4 ו- 4 סוגים של משאבים: A, B, C, D
נתון כי התהליכים מחזיקים כרגע במשאבים על פי הטבלה הבאה:

Current allocation matrix:

P_1	1	1	1	2
P_2	2	1	2	0
P_3	1	2	1	1
P_4	1	0	1	1

נתון כי מקסימום הדרישות העתידיות האפשרי של אותם תהליכים הוא:
Request matrix:

P_1	2	4	3	2
P_2	5	3	4	2
P_3	2	2	2	2
P_4	1	0	2	1

ונתון כי המשאבים הפנויים עדיין הם (Available Resource Vector) (1,1,1,2) :

א. (10 נק')
האם המערכת במצב לא בטוח? אם כן, אילו תהליכים עשויים להיות בקיפאון (deadlock)?

ב. (5 נק')
מהם הערכים המינימאליים של המשאבים הפנויים שעבורם עדין נכון כי המצב הוא safe (לא יתכן deadlock במערכת)?

<u>סעיף א:</u>

המערכת במצב לא בטוח.

ניתן לראות שהמשאבים הפנויים 1,1,1,2 לא יכולים לספק אף תהליך מה request matrix

ז"א כל התהליכים יכנסו לקיפאון כי אין לנו יכולות להתקדם.

לדוגמה - חישוב על הסבב הראשון, התהליכים שעשויים להיות בקיפאון הם:

שלב ראשון נקצה הכל ל :

P1 + A = 2,2,2,4 אין מספיק משאבים - עשוי להיות בקיפאון

שלב שני נקצה הכל ל:

P2 + A = 3,2,3,2 אין מספיק משאבים - עשוי להיות בקיפאון

שלב שלישי נקצה הכל ל:

P3 + A = 2,4,2,3 יש מספיק משאבים

שלב רביעי נקצה הכל ל:

P4 + A = 2,1,2,3 יש מספיק משאבים

<u>סעיף ב:</u>

הערכים המינימאליים של המשאבים הפנויים הם לפחות 2,2,2,1 בווקטור הפנוי Available במצב כזה נוכל לרצות את התהליכים:

P4-> P3 -> P1 -> P2

שאלה 4
הושלם
5.00 נקודות
מתוך 5.00
סימון שאלה

במערכת הפעלה שתומכת ב- multi-threading רצים שני תהליכים במקביל: A ו- B. התהליך הראשון (A) מריץ 3 חוטים (threads) במקביל: A_1, A_2, A_3 והתהליך השני (B) מריץ 2 חוטים במקביל: B_1, B_2.

ציינו נכון/לא נכון לכל אחת מהטענות הבאות:

א. A_1 ו- A_2 משתמשים במחסניות נפרדות.

ב. על מנת לחלוק במשתנים משותפים, A_1 ו- A_2 חייבים ליצור זיכרון משותף (shared memory) בעזרת מערכת הפעלה.

ג. על מנת להשתמש במשתנים משותפים, A ו- B חייבים ליצור זיכרון משותף (shared memory) בעזרת מערכת הפעלה.

<u>סעיף א:</u>
נכון.
משתמשים בפועל במחסניות נפרדות אך מקושרות לאבא אחד.

<u>סעיף ב:</u>
לא נכון.
מכיוון שהם תהליכונים של אותו אבא יש להם זיכרון משותף שכבר מוגדר להם.
כמו כן בנוסף אפשר להגדיר ביניהם pipe.

<u>סעיף ג:</u>
נכון.
בגלל שאין קשר בין תהליך A לתהליך B ולכן נצטרך לקשר ביניהם כדי שהם יוכלו לשתף מידע.

שאלה 5
הושלם
5.00 נקודות
מתוך 5.00
סימון שאלה

נתון קטע קוד הבא:

```
int n=0;
void InterruptHandler(int signum){
   signal(SIGINT,InterruptHandler);
   printf("Received %d, value of n is %d\n", signum, n);
   while (n<20) {
         n+=2;
         sleep(1);
   }
}

main(int argc, char **argv){
   signal(SIGINT, InterruptHandler);
   while (n<20) {
         n++;
         sleep(1);
   }
}
```

ענו על השאלות הבאות:

1. אחרי כמה זמן התוכנית תסתיים מעצמה ומה תדפיס?

2. איך ומתי ככל שמוקדם ניתן לסיים את התוכנית לפני הזמן ומה תהיה ההדפסה?

<u>סעיף 1:</u>

בהתחלת קטע הקוד n = 0

מקרה 1:

ברגע שקיבלנו סיגנל SIGINT - ביצוע של CTRL+C אנו נכנס לפונקציה של interrupt

שם ישנה לולאת while מגדילה כל פעם את n = n+2 עד אשר n < 20

וכל איטרציה הוא מחכה 1 שניות עד ש n מגיע ל 20 ואז חוזרים ל main

ומכיוון ש n הוא כבר 20 אז הוא לא מבצע בכלל את ה while שנמצא ב main.

לכן סה"כ אחרי 10 שניות + זמן תקורה מועט של ריצת הקוד.

התוכנית תדפיס printf("Received %d, value of n is %d\n", **signum**, **n**)

signum הוא בעצם מספר הסיגנל SIGINT שהוא סיגנל של ביצוע CTRL+C = מספר סיגנל הוא 2

והמספר של **n** שהוא 0 בהתחלת התוכנית

"Received 2, value of n is 0"

מקרה 2:

כאשר אין סיגנלים כלל אז אין קריאה לפונקציית interrupt ומתבצע רק ה while בפונקציית ה main ולכן סה"כ נרוץ 20 שניות + זמן תקורה מועט של ריצת הקוד.

התוכנית לא תדפיס כלום במצב כזה.

מקרה 3:

יכול להיות גם תרחיש גרוע שישנם כמה קריאות סיגנלים של SIGINT ברצף ואז צריך כל פעם לעצור את הסיגנל שרץ כרגע ולטפל בסיגנל החדש.

זמן הריצה המקסימלי יהיה 10 שניות כי סה"כ ה sleep מתבסס על ערכו של n.

וכל פעם יודפס **n** שונה בהתאם לאיפה שנעצרה הלולאה של הסיגנל הקודם.

signum הוא עדיין 2 כל פעם

<u>סעיף 2:</u>

כאשר ישנו סיגנל אחד בודד שמסתיים.

סה"כ אחרי 10 שניות + זמן תקורה מועט של ריצת הקוד.

התוכנית תדפיס printf("Received %d, value of n is %d\n",**signum**,**n**)

signum הוא בעצם מספר הסיגנל SIGINT שהוא סיגנל של ביצוע CTRL+C = מספר סיגנל הוא 2

והמספר של **n** שהוא 0 בהתחלת התוכנית

"Received 2, value of n is 0"

הערה:
למה גרוע?

שאלה 6
הושלם
3.00 נקודות
מתוך 5.00
סימון שאלה

אילו מהטענות להלן נכונות עבור מערכות קבצים מבוססת על i-nodes.

1. מקצה בלוקים ברצף.

2. ניתן לשמור את התוכן של קבצים קטנים ישר בתוך ה i-node

3. גודל המקסימאלי של קובץ מוגבל.

4. מחזיקה מבנה נפרד (כגון bitmap) עבור מעקב אחרי בלוקים פנויים בדיסק.

5. ניתן להקצות מספר (גדול מ-1) i-node -ים עבור קובץ אחד.

1:
לא נכון.
מערכות קבצים INODES עובדות עם מצביעים ולכן לא אכפת להן איפה הבלוקים נמצאים בזיכרון אבל בתוך ה INODE היא כן שומרת אותם ברצף.

2:
לא נכון.
יש קישורים ישירים הקובץ עצמו לא נשמר ב INODE ז"א הוא מכיל רק את המאפיינים של הקובץ ולא את המידע שלו.

3:
נכון.
הוא מוגבל אבל לגודל מאוד גדול ואם רוצים אפשר להגדיל ע"י הורדת מצביע ישיר והוספת single, double, triple..
אבל הוא עדיין מוגבל.

4:
לא נכון.

5:
נכון.
אם הקובץ גדול מאוד אז נדרש להרבה inodes

הערה:
1 לא נכון
2 לא נכון
3 נכון
4 נכון
5 לא נכון

שאלה 7
הושלם
4.00 נקודות
מתוך 5.00
סימון שאלה

איזה פעולות צריך לבצע כדי לממש את הפקודה mv /a/b/c /d/e שמעבירה קובץ ממקומו המקורי למקום אחר. תתייחסו ל 2 מקרים: כשקיימת תיקיה בשם e וכשאינה קיימת.

<u>מקרה 1:</u>
כאשר קיימת תיקיה e
קודם כל יש ליצור קובץ חדש בתיקיה e
ואז יש צורך להעתיק את כל המידע שיש בקובץ המקורי לקובץ שעכשיו יצרנו בתיקיה e
וכעת למחוק את הקובץ המקורי
או שניתן לשנות את הקישורים של הקובץ המקורי שלב אחרי שלב:
קודם כל לנתק מהתיקייה c ואז לקשר לתיקיה e

<u>מקרה 2:</u>
כאשר התיקייה e אינה קיימת
יש תחילה ליצור את התיקייה e ולקשר אותה לתיקיית האב d ואז ניתן לעשות כל מה שהיה במקרה 1, יש ליצור קובץ חדש בתיקיה e
ואז יש צורך להעתיק את כל המידע שיש בקובץ המקורי לקובץ שעכשיו יצרנו בתיקיה e
וכעת למחוק את הקובץ המקורי
או שניתן לשנות את הקישורים של הקובץ המקורי שלב אחרי שלב:
קודם כל לנתק מהתיקייה c ואז לקשר לתיקיה e

הערה:
היה צריך להתייחס לפעולות פרטניות, כמו קריאת i-node ואז תוכן...

כש e לא קיימת: לא צריך ליצור, הקובץ יעבור בשם E

שאלה 8
הושלם
5.00 נקודות מתוך 5.00
סימון שאלה

איזו מההצעות הבאות אינה שיטה אפשרית למניעת deadlock:

א) דרישה שכל תהליך יימנע מלבקש משאבים בצורה שתגרום ליצירת מעגל בגרף ההקצאות

ב) קביעת יחס סדר על המשאבים ודרישה שתהליך המחזיק במשאבים מסוימים יכול לבקש רק משאבים בעלי מספר גבוה יותר

ג) דרישה שכל תהליך יבקש את כל המשאבים שלו בבת אחת בתחילת הריצה

ד) דרישה שאם תהליך רוצה להשיג משאבים נוספים, הוא צריך קודם כל לשחרר את אלה שהוא כבר מחזיק

הצעה א:
מונעת deadlock

הצעה ב:
הצעה זו אינה מונעת deadlock
כי אם נניח נגיע למצב שבו כל התהליכים זקוקים למשאב מדירוג יותר נמוך ממה שיש לו אז נגיע למצב של קיפאון כיוון שאף אחד מהתהליכים לא יוכל לבקש משאב כזה.

הצעה ג:
מונעת deadlock

הצעה ד:
מונעת deadlock

שאלה 9
תקין
5.00 נקודות
מתוך 5.00
סימון שאלה

ה - working set של תהליך היא קבוצה של דפי זיכרון בעלת התכונות הבאות:

יש לבחור תשובה אחת:

- הדפים בקבוצה נבחרו על ידי אלגוריתם השעון.
- הדפים בקבוצה מאורגנים לפי סדר הגישות האחרונות לזיכרון.
- הדפים בקבוצה הם המועמדים להעברה לדיסק כשיש page fault וצריך לפנות מסגרות.
- גודל הקבוצה משקף את מידת הלוקאליות של התהליך בגישות לזיכרון. ✓

תשובתך נכונה.

שאלה 10
תקין
5.00 נקודות
מתוך 5.00
סימון שאלה

הזמן לביצוע קריאת מערכת read(fd, buf, 100) ראשונה מקובץ שנפתח קודם תלוי בעיקר,

יש לבחור תשובה אחת:

- בזמן הדרוש להעתיק את הנתונים מ buffer cache לזיכרון של המשתמש.
- בזמן של קריאת הנתונים מהדיסק. ✓
- בזמן לבצע את קריאת המערכת עצמה, כלומר פקודת trap
- בשיטה שמשתמשים בה (DMA או PIO).

תשובתך נכונה.

שאלה 11
תקין
5.00 נקודות
מתוך 5.00
סימון שאלה

בהשוואה בין VMM (virtual machine monitor) בתצורת bare metal ובתצורת hosted, מהו יתרון של תצורת hosted?

יש לבחור תשובה אחת:

- שליטה מוחלטת על משאבי החומרה.
- אפשרות הרצת 2 מערכות הפעלה שונות במקביל.
- הרצה של אפליקציות משתמש מחוץ ובמקביל למכונה וירטואלית ✓
- ביצוע יעיל יותר של פקודות מערכת (system calls).

שאלה 12

תק'ין

5.00 נקודות
מתוך 5.00

סימון שאלה

האם ניתן לממש את אבטחה בשיטה "רשימת יכולות" (C-LISTS) במרחב כתובות של משתמש?

יש לבחור תשובה אחת:

- כן, בעזרת מנגנון חומרה מיוחד שמאפשר לקבוע לגבי כל איזור זיכרון ראשי האם ניתן לכתוב ✓
- לא, בגלל שאז מערכת הפעלה לא תוכל לשנות אותן.
- לא, כי אז המשתמש עצמו יוכל לשנות את היכולות שלו.
- כן, כי אחרי שיכולות נקבעו ע"י המערכת תהליך משתמש לא יוכל לשנות אותן.

תשובתך נכונה.